Native Six-Degree-of-Freedom Rendering on Lightweight See-Through Glasses Without a Game Engine

An implementation on RayNeo X3 Pro that reaches the head tracker from outside the vendor toolchain, with on-device measurements and an analysis of field use.

Spatial Computing and Wearables Group, KAUST, with AKULAR and Makani Parking.

Prepared under the programme “Interacting with Digital Twins Through AI-Powered Augmented-Reality Hardware.”

Consortium Technical Report · pre-publication draft · 2026.

Abstract

A head-worn display becomes useful to a field crew once a rendered mark holds its position on a real object while the wearer walks around it. Producing that behaviour on lightweight see-through glasses is hard. The public interface on these devices returns head orientation and keeps head position private. Both eye images share one physical panel, and its waveguide optics defeat a first attempt at stereo calibration. A device worn on the face carries little compute and a small battery. The common route to registered content passes through a commercial game engine and a vendor plugin, which draw power the glasses can barely spare and hide the tracker behind a managed runtime.

This report describes a working implementation on RayNeo X3 Pro binocular glasses. A signed third-party application, with no game engine linked, draws three-dimensional content that keeps its place in a room as the wearer circles it. Reaching that point required access to the device’s six-degree-of-freedom head pose from outside the sanctioned toolchain, recovery of the coordinate convention its tracker reports in, a stereo projection derived from the device’s own optical constants, and presentation through a standard Android surface. We give an account of the engineering, including the class of faults that return well-formed and incorrect answers, the acceptance procedures that separate them, and the processor and graphics load of the resulting client. We then follow the same capability from parking enforcement into maintenance inside dense data halls, with a clear line between what the hardware does today and what the wider programme will build.

Keywords: augmented reality; registration; visual-inertial odometry; head tracking; optical see-through displays; stereo rendering; digital twins; edge computing; infrastructure inspection.

1. Introduction

Ask a maintenance technician where the day goes, and a large share of it turns out to be spent carrying a record back and forth to an object. A value is read on a handset. The technician looks up at a valve, a vehicle, a rack. A judgment forms. The head drops again to enter the result. Each of those crossings costs a few seconds, and each opens the door to a wrong entry or a mistaken identity.

A display worn on the head can put the record on the object and leave both hands free. The idea is as old as the first augmented-reality prototypes [13]. What has stayed hard is making it behave on glasses light enough to wear through a shift.

One behaviour carries the whole idea. A rendered mark has to appear at a chosen point in the room and stay there as the wearer moves. The augmented-reality literature calls this alignment registration [13,14]. Good registration asks the device to report, tens of times a second, where the head sits and where it points, and it asks the renderer to draw from that viewpoint quickly enough that the eye never catches the mark drifting. A phone held at arm’s length forgives a great deal of slack. Glasses worn in front of the eyes forgive very little.

Our consortium is working toward outdoor and industrial deployments in Saudi Arabia: kerbside parking in the old city of Jeddah, bilingual heritage guidance, and bounded inspection at airport and industrial sites. In the full system, speech, a live digital twin, and a mobile robot sit over one shared coordinate frame. This report covers the component the rest of them require, stable registration on lightweight glasses with no game engine in the build. We give the result, the reasons it resisted an easy solution, the numbers we have, and an honest account of what is left.

2. Engineering Contribution

This section states the result in engineering terms and marks its edges. Later sections expand each point.

2.1 The problem

RayNeo X3 Pro glasses ship with a documented interface that returns head orientation. Orientation on its own drives a heads-up panel that floats with the gaze. It cannot pin a mark to a physical object, because pinning needs translation: a step to the side must move the mark in the view by a matching amount, and orientation carries no such information. The device runs a full six-degree-of-freedom tracker inside a system service. Third-party developers were

expected to reach it through Unity and a vendor plugin, and no public path to it existed outside that toolchain.

2.2 Why it was hard

The difficulty came from four directions at once, and each one has a different character.

The position stream lives behind a native service that the public interface never advertises, so recovering it took reverse engineering of the runtime. The failures in this layer are abrupt. A wrong binding to the runtime crashes the process with no diagnostic, and a wrong configuration value is accepted in silence and changes nothing.

The tracker reports its pose in a convention it never states. Reading that convention wrongly still produces a scene that renders, responds to the head, and looks correct under the motions a person naturally tests with. The errors in this layer are quiet, and they survive casual testing.

The panel carries both eye images on one physical surface, through waveguide optics that route the halves in a way a first guess gets wrong. With no engine present, the per-eye projection had to be derived from the device's own calibration constants and confirmed on hardware.

The compute and thermal limits of a face-worn device leave little room for a managed engine runtime to sit beside the tracker and the two displays. Power consumption became a design input in this work, at the same level as correctness, and it decided several of the choices in Section 5.

2.3 What the work demonstrates

A signed third-party application, with no game engine linked, obtains the six-degree-of-freedom head pose, computes a per-eye stereo projection from the device optics, and draws content through an ordinary Android surface. Rendered geometry stays fixed in the room as the wearer walks around it, and returns to the same coordinates after the wearer leaves the area and comes back, which points to a globally consistent map.

2.4 Why it is worth reporting

Registered rendering on this class of glasses no longer needs a commercial engine or the vendor's sanctioned toolchain. A native path takes a managed runtime off a power-limited device and shortens the route from a pose estimate to a drawn frame, which is the route that decides whether content sits still. Every application the programme targets, from enforcement to inspection, reuses this same capability and changes only the recognition model and the record it writes to.

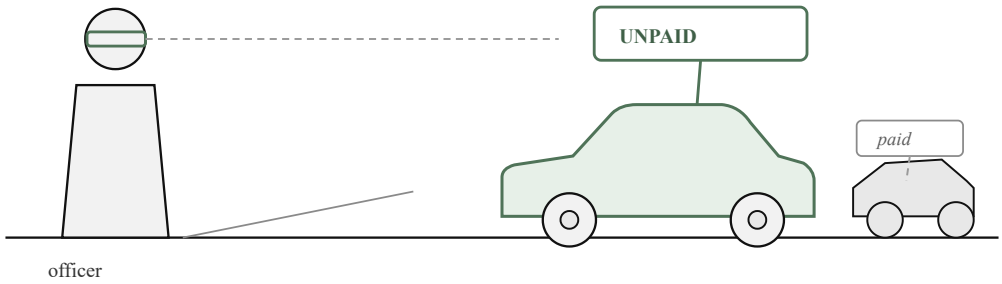


Figure 1. An enforcement officer sees each vehicle's payment status pinned to the car itself. The flagged vehicle carries its record where the officer is already looking, so identity and status arrive in the same glance.

3. Background and Related Work

Head-mounted displays fall into two families. A video pass-through headset renders the world onto interior screens, so software owns every pixel the wearer sees. An optical see-through device places a small transparent combiner in the line of sight and adds light to a world the wearer views directly. The glasses here are the second kind. Software controls only the added light, and the real scene never reaches the frame buffer.

Position tracking on such hardware grew out of two decades of visual and visual-inertial estimation. Early real-time monocular systems set the shape of the problem [1,2]. Feature-based pipelines matured into ORB-SLAM and its successors [3,4]. Tight fusion of a camera with an inertial unit, through filtering [5] or through optimisation with preintegrated inertial terms [6,7], produced the accuracy and the tolerance for fast motion a head tracker needs; direct methods followed a parallel line [8]. RayNeo's runtime implements a system of this family and exposes it to Unity developers through its ARDK [20], and the interface it presents follows the conventions of the OpenXR runtime model [16]. Our work touches none of the estimation mathematics. We recovered a path to the existing estimator from outside the sanctioned toolchain and built a renderer around it.

Drawing correctly into see-through optics is a separate literature. Projection from a calibrated camera follows the standard pinhole treatment [9], with wide-angle terms where they are needed [10]. Aligning rendered content to a user's eye through a combiner has been studied since the single-point active alignment method, and later surveys catalogue the calibration approaches and their error sources [11,12]. The perceptual stakes are well recorded: both registration error and latency break the sense that content belongs to the scene [13,14].

Latency here carries a hard physical meaning, the time from head motion to the matching photon, and cutting it has driven dedicated display research [15].

The operational half of the programme draws on digital-twin practice for infrastructure [21,22] and on recent perception models for the recognition tasks that will sit over tracking, among them open-vocabulary image models [23], promptable segmentation [24], and multilingual speech recognition [25]. Visual positioning services such as ARCore Geospatial, Azure Spatial Anchors, and Immersal address the related outdoor-relocalisation problem and informed the wider design [26].

> **WHY SATELLITE POSITIONING IS NOT ENOUGH**
>
> A satellite fix locates a receiver to within several metres and says nothing about heading. Registration needs position good to a few centimetres and orientation good to a fraction of a degree. One axis is off by two orders of magnitude; the other is absent.
>
> Fixes also arrive slowly, drop indoors, and wander near the large metal surfaces of parked vehicles. A person turns their head faster than a receiver updates. For a mark that has to stay on an asset while the head moves, satellite positioning gives a coarse hint and no more.

4. The Combined Tracking Problem

Any one difficulty at a real site has a known countermeasure when it arrives alone. They never arrive alone. They land on different stages of one pipeline, and because each stage feeds the next, the effects compound and the total is worse than the sum of the parts.

Picture a parking area at midday. The camera rides a head that turns quickly, which smears the image. The sun moves and drops hard shadows, so exposure swings between frames. Asphalt is a broad, repeating texture with little for a tracker to hold. Car bodies are glossy and mirror the sky, so they show bright features that belong to a reflection and to no fixed surface. Vehicles come and go, so the scene will not settle. Many of them look alike, so telling one from another at a fresh angle is genuinely uncertain.

Trace a single failure along the chain. A visual-inertial tracker reads head motion from image features. Fast motion blurs them, blur weakens the motion estimate, and a weak estimate pulls the anchor off its true point. The mark slides, the wearer moves to keep it in view, and the added motion feeds more blur back to the tracker. A reflection hands the estimator a confident feature that sits nowhere real, so the estimator trusts it and drifts toward it. A passing car removes the ground features the tracker was leaning on at the moment it is asked to hold. Failures of this kind couple through the environment, and bench numbers rarely survive the move outdoors.

> **WHY PARKING AREAS ARE CLOSE TO A WORST CASE**
>
> The ground is a large, flat, repeating surface with few stable landmarks. The main objects are mirror-finished and reflect a moving sky. The scene changes constantly, because the vehicles are both the content and the clutter. Look-alike vehicles leave identity from a new viewpoint uncertain.
>
> A working lot also refuses to be scanned under kind conditions. Nobody empties it and maps it at leisure. The system has to localise in the same crowded, shifting scene it is meant to annotate.

Deciding what an object is, from one frame, is a classification that current vision models handle well. The harder task runs in time. A vehicle has to stay bound to one physical point across changing viewpoints, through partial occlusion, and across the moments when the tracker loses its place and recovers. Recognising an object once is a judgment made and finished; holding a mark on the right object, in the right place, through minutes of motion is a judgment defended every frame. A better single-frame classifier will not settle it.

The usual answer to a hard perception task is more computation, and face-worn glasses take that answer away. The camera sits on a fast-moving head, so the input is harder than a handheld feed. Compute and battery stay small, because the device has to remain light and cool enough to wear across a shift. The see-through panel offers a narrow field of view through optics with little tolerance, so a registration error has nowhere to hide. These limits set the terms for everything in the next section.

5. Implementation

Six pieces of work stand between a device that reports orientation and a scene that holds its place in a room. They are described here at the level a reader needs to recognise the same obstacles on other hardware. Names of vendor libraries and symbols, dispatch offsets, and the encodings of configuration values are held back by intent, and none of them is needed to follow the argument.

5.1 Reaching the position stream

The public interface returns orientation and keeps position private. The device’s own six-degree-of-freedom tracker runs inside a system service, and we established that an ordinary signed application can connect to that service as a client and read its output. What earlier accounts had treated as reachable only through the vendor’s engine turned out to be reachable from a self-signed application, and the position stream now feeds the renderer at the rate the application samples it.

Calling the entry point safely required more care than locating it. It dispatches through a table of function pointers that the runtime fills during its own initialisation. An application that

resolves the symbol against a private copy of the same library receives a valid handle and a table that is entirely empty, and the first call walks into a null entry and takes the process down with no diagnostic. The resolution has to bind to the copy the runtime itself initialised. Our client therefore resolves only against a mapping already present in the process, checks that the dispatch table is populated before every call, records which file the resolved symbol came from, and returns a distinct error for each way the check can fail. A failure that would otherwise terminate the process without explanation is reported as a log line naming the cause.

Switching the tracker on proved insufficient by itself. The runtime also has to be told which attitude algorithm to use, through a configuration value whose accepted encoding is not discoverable by inspection and whose obvious candidates are silently ignored. An out-of-range value produces no error and no effect. The tracker keeps running, the orientation channel keeps updating, and the position channel keeps returning a small residual displacement synthesised from head rotation, which is convincing enough to survive a casual look. Measurement identified it: a wearer walking one to two metres produced a position span of about three centimetres, and the displacement swung with head pitch and returned when the head returned, which is the signature of a lever-arm model driven by orientation. The client writes the value and reads it back before trusting it.

A third instance of the same pattern is worth recording. The vendor’s own published binding for the pose call declares an argument list that the shipped implementation does not use; the real call takes an additional argument and orders the others differently. Called as published, it never writes the position at all and returns an orientation sampled at a meaningless time, so the output is finite, correctly normalised, and uncorrelated between frames. Read without suspicion, that looks precisely like a tracker that has not yet localised. A published interface description was wrong in a way that passed every cheap sanity check.

> **THE RULE THAT CAME OUT OF THIS LAYER**
>
> On an undocumented runtime, the absence of an error is no evidence of an effect. Every call that changes state is written and then read back, and health is judged from the tracker’s own reported state, whose meaning we had to correct first: the value we initially read as success turned out to mean that initialisation was still in progress, so a tracker that never started looked healthy for some time.

5.2 Recovering the pose convention

The tracker reports a rotation and a translation and states the meaning of neither. Whether the rotation carries the head into the world or the world into the head, which handedness applies, which axis points where, and whether the translation lives in the same frame as the rotation are all left open. Every combination renders. Most of them respond to head motion in a way that looks broadly correct.

We enumerated the candidate conventions, cycled them on the device, and had a wearer describe the behaviour of a drawn set of world axes. That produced a convention that passed

its tests. It was wrong, and the way it was wrong is the most instructive result in this report.

The test motions were degenerate. Under pure yaw, and under pitch while facing an anchored object, the convention we chose and the correct one give the same visible result. The two motions that separate them are head roll, and pitch performed while facing away from the anchor. Neither appears in the natural test of placing an object, walking up to it, and walking back. Both errors appear only under those motions: tilting the head sideways rotated the scene about the view axis at twice the true rate, so content that should have stayed level with the floor did not, and after walking through a scene and turning around, head pitch moved the content in the wrong direction.

Both symptoms follow from a single error. The reported rotation is expressed in a frame reflected in one axis with respect to the render frame, so the correct transform is the reported rotation conjugated by that reflection, with the same reflection applied to the translation. A conjugation of this form is invisible under symmetric motion and unmistakable under asymmetric motion, which is why the first convention survived its acceptance test. The correction preserved every earlier validated observation, resolved both new symptoms at once, and brought the rotation into agreement with a translation sign that had been settled independently, by the observation that anchored content fled the wearer on approach when the sign was inverted.

The practical lesson generalises to any integration with an undocumented pose source. Acceptance tests have to include head roll and motion performed while facing away from the anchor, from the first day. A test suite built from the motions people naturally perform will certify a whole class of frame errors as correct.

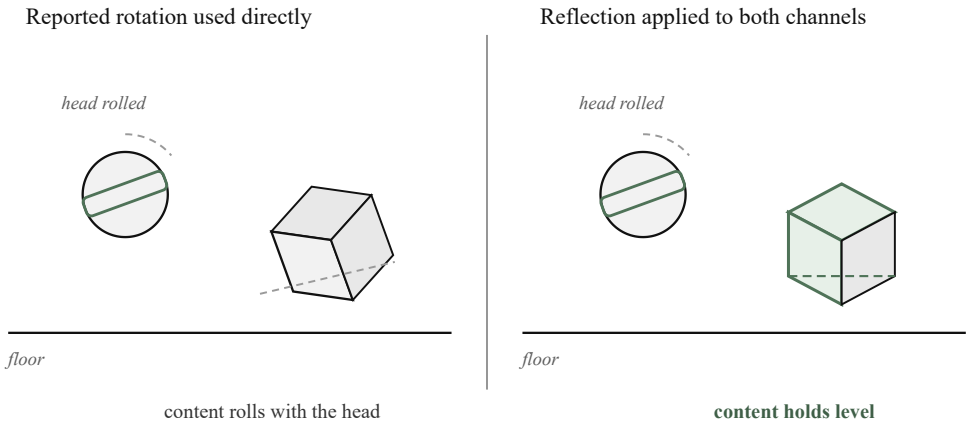


Figure 2. The failure that exposed the frame error. Under head roll, the first convention rotated anchored content about the view axis at twice the true rate, while yaw and forward pitch had looked correct throughout. Conjugating the reported rotation by a reflection in one axis, and applying the same reflection to the translation, corrected both this symptom and a reversal of pitch that appeared only when the wearer faced away from the anchor.

5.3 Per-eye projection from the device optics

An engine hands the developer a ready per-eye camera. With none present, the projection had to be built from what the device publishes. The calibration arrives in computer-vision form, as focal lengths and a principal point in pixels for each eye at a resolution of 640 by 480, which corresponds to roughly 24 degrees of horizontal field of view. No projection matrix is supplied. The vendor's eye-to-render transform is the identity for both eyes, so the supplied calibration carries no interpupillary baseline.

The glasses present themselves to the operating system as one ordinary landscape surface. That surface is physically a side-by-side stereo buffer, with one half driving each eye through waveguide optics that route the halves in a way a first assignment gets wrong. The renderer draws one frame in two passes, setting both the viewport and the scissor rectangle to the half being drawn, so that the clear as well as the geometry stays inside one eye.

The projection is assembled directly from the intrinsics, holding the computer-vision camera convention in which the view axis points forward and the image ordinate increases downward, and folding the conversion to the display's normalised device coordinates into the matrix. Because the principal point sits at the centre of each half, the matrix carries no off-axis terms; a device whose principal point were offset would need them, and this is the point at which a general off-centre frustum becomes necessary. Both matrices depend on the surface size and on constants, so they are computed once when the surface is created and never touched again.

Two-dimensional overlay content follows the opposite rule. The same content is drawn into both halves, which makes the routing question irrelevant and places the overlay at optical infinity, where it carries no vergence conflict. Safe-area insets are expressed as a fraction of one half, since the field of view vignettes at the edges and an inset measured against the full surface would land in the wrong place.

The current build renders both halves from a single viewpoint, which fuses correctly at conversational distance and understates depth for near-field content. The remedy is a fixed per-eye offset along the head-right axis, and it is a contained change.

5.4 The render path

Content reaches the panel through a standard Android surface, drawn with OpenGL ES [17] and presented on the display's own vertical-sync signal through the platform graphics stack [18]; the native code runs under the Android NDK [19]. No managed engine runtime, no engine scene graph, and no engine frame scheduler sit in the path.

Static scene content is transformed into world coordinates once, when it is loaded, and uploaded as a single vertex buffer. A frame then costs one draw call per eye and one matrix upload, with no scene-graph traversal and no per-object transform. The only per-draw transform in the system belongs to loaded models.

One consequence of the projection derivation deserves recording, since it recurs on any device whose pose convention requires an axis flip. The view transform contains a reflection, so its

determinant is negative and the apparent winding of every triangle reverses. While the renderer sorted faces by depth and drew all of them, as the first software implementation did, winding never mattered and the geometry was allowed to carry an inconsistent set of face orderings. Enabling back-face culling for power reasons made winding decisive overnight. Two changes were needed together: the geometry was rewound into one consistent outward orientation, and the renderer was told that clockwise triangles face front, which compensates for the reflection. Either change on its own renders the scene inside out. With both in place, roughly half of the triangles in the test scene never reach the rasteriser.

5.5 Holding the power budget

A conventional renderer draws on every display refresh. In this application the scene is static and the only input is the head pose, so a frame that would repeat its predecessor has no reason to exist. The surface therefore renders on demand, and a frame is requested only when a newly sampled pose differs from the previous one beyond a small threshold. Pose sampling runs at about 30 Hz on a background thread, so the render rate follows the pose rate and drops further when the wearer holds still.

The measured effect on hardware was a fall from 60 frames per second, locked to the panel, to 28.6 frames per second driven by the pose, with the graphics unit busy about 2.7 per cent of the time and the application consuming about 3 per cent of one processor. The tracking service, at about 12 per cent, remains the dominant term and is outside our control. On a device with this thermal budget, the work the application declines to do is worth more than any optimisation inside the work it performs.

Three supporting measures carry the same intent. The draw path allocates nothing: every transform buffer is preallocated and reused, so the garbage collector never runs on the render thread. Constant work is hoisted out of the frame, which is why the projection matrices are built once per surface. Diagnostic instrumentation is compiled out of the shipping configuration, and the overlay layer is omitted entirely when it carries nothing, so the compositor handles one layer in place of two; making a view invisible would have left its cost in the compositor.

> **WHY THE MOTION-TO-PHOTON INTERVAL GOVERNS STABILITY**
>
> The eye judges registration by one question: does the mark travel with the head or stay with the room. The measurable quantity behind that judgment is the interval from a head movement to the arrival of the matching photon on the panel. A long interval lets the mark lag the head and slide across the scene, and it provokes discomfort, since the eyes and the inner ear then disagree.
>
> Every stage between the pose estimate and the drawn frame adds to this interval. Taking out the engine takes out stages. That is the direct link between a lean presentation path and a mark that holds where it was placed.

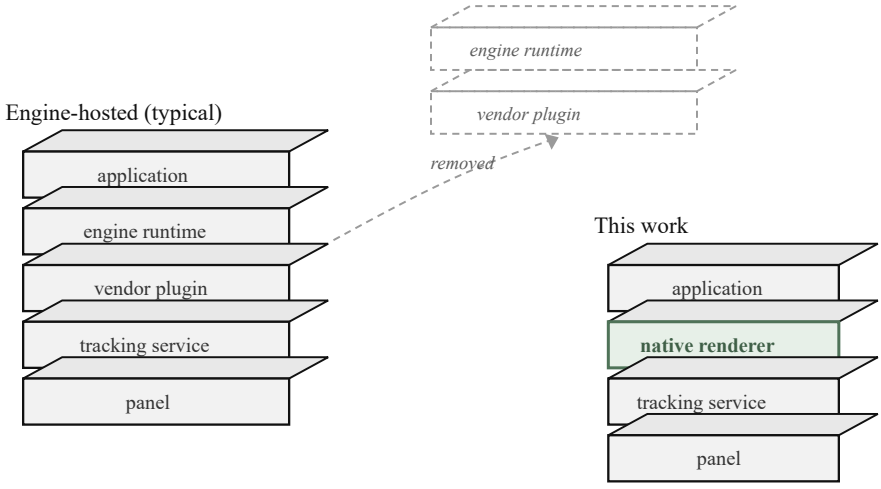


Figure 3. The presentation path with the game engine and its vendor plugin lifted away. Removing two software layers shortens the route from a head-pose estimate to a displayed photon and lowers the resident cost on a thermally limited device.

5.6 Content and build path

Removing the engine removes its asset pipeline as well, so the renderer reads the standard binary glTF container directly [27], in about 300 lines that use only the platform's own JSON parser and image decoder. Node transforms are baked into the vertices at load time, so the draw path needs no hierarchy; normals receive the inverse-transpose treatment and are generated from face geometry when an asset omits them. Unsupported features are refused with a message in place of a silent approximation.

Two limits of this class of hardware surface directly in that loader, and an engine would have hidden both. The graphics profile carries no 32-bit index support, so wider index buffers are accepted only when every value fits the narrower type and are rejected otherwise. Mipmapping and texture repeat require power-of-two dimensions, so textures that fall outside that constraint have their filtering and wrap modes downgraded at load, in place of producing an incomplete texture that would render black.

The application builds with platform tools alone, with no engine, no build daemon, and no network access, and it installs as an ordinary Android package. For deployments that have to certify what a device does, an artefact of this size that can be read end to end is a practical asset in its own right.

6. System Architecture

The tracker and renderer live inside a larger system laid out along two axes: how quickly a result is needed, and how sensitive the data is. Urgent or private work sits close to the wearer; only optional, non-sensitive work moves outward.

The glasses hold the head pose and the render surface, the demonstrated part of this report. The wearable owns the estimate and draws the registered content, with no engine in the path. A companion phone runs light detection and the interaction shell, carrying the speech layer and short-term task memory and routing requests to the right tier. An on-premises edge server runs heavy vision, hosts the digital twin, and keeps the audit log; writes from the field land there with a full trail. Cloud service stays disabled by default and handles only non-sensitive batch work, and then only when the latency budget allows. Sensitive streams do not cross the on-premises boundary.

One shared coordinate frame ties the tiers together. A flag a person raises and a waypoint a robot plans name the same point in the same room, because both come from the same head-tracked frame. That common frame lets a person and a machine work as one team over a single set of points. Without it, each would hold its own coordinates and the two would work past each other.

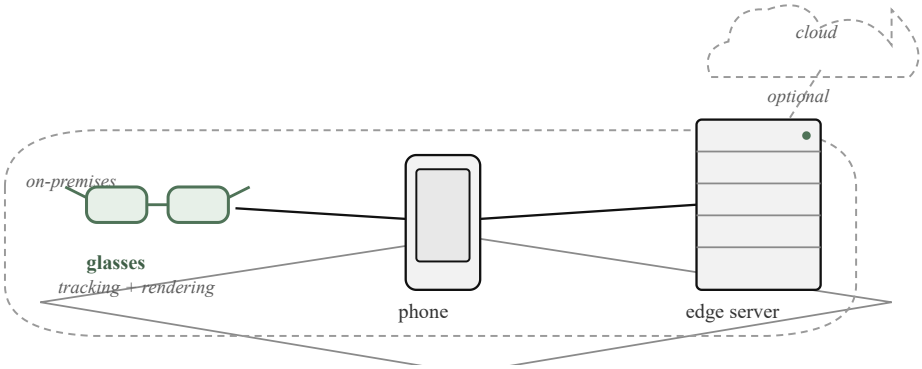


Figure 4. Where the system runs. Tracking and rendering stay on the glasses; the phone adds light detection and speech; an on-site server holds the digital twin and the audit log. Sensitive data stays inside the on-premises boundary, and cloud service is optional.

7. Operational Example: Parking Enforcement

We walk one realistic task against the demonstrated capability and mark each step by its status. The setting is enforcement in a public parking area, the scene sketched in Figure 1.

An officer wearing the glasses approaches a row of parked cars. Registration is live, so anything the system draws will keep its place as the officer moves. *(Demonstrated: tracking and stable rendering.)* The vision layer identifies vehicles in view and reads identifying detail, a single-frame judgment. *(Operational target, built on established vision models.)* Each recognised vehicle is fixed to a point in the area's coordinate frame, so a mark placed on it will hold as the officer passes. *(Placement uses the demonstrated tracking; binding recognition to it is integration work.)* The system checks the vehicle's identity against the parking record and reports whether it has paid. *(Operational target: a query to the back-office record.)*

An unpaid vehicle receives an outline and a short status label, drawn on the car and held there as the officer walks. *(Rendering demonstrated; the content it carries is the target.)* The officer looks at the car and asks for detail by voice, and the record appears in place. *(Operational target: speech and twin layers.)* A spoken word or a gesture confirms the finding; the person makes the call. *(By design, the operator decides.)* The action writes to the digital twin with an audit entry and reaches back-office systems within seconds. *(Operational target: twin writeback and audit.)* The officer moves to the next car and repeats. Several vehicles can carry marks at once, so a full row can be read in a glance.

Each step carries real difficulty. Recognition has to work through sun, shadow, reflection, and partial occlusion on a compute-limited device; localisation has to place the mark within a few centimetres so it lands on the intended car and not its neighbour; validation has to match the correct record to the correct physical object among look-alikes; and the log has to be trustworthy enough to stand as a record. The demonstrated capability answers the one step the others cannot proceed without.

> **DEMONSTRATED TODAY VERSUS OPERATIONAL TARGET**
>
> Demonstrated on hardware: recovery of the six-degree-of-freedom head pose from a third-party application, a stereo projection derived from the device optics, engine-free rendering through a system surface, and registered content that holds position as the wearer circles it.
>
> Operational target of the programme: vehicle recognition and identity, validation against records, twin writeback with audit, and the speech layer. These extend the demonstrated capability with established components, and this report does not present them as complete.

8. Applications

Registration does not care whether the anchored object is a car, a container, or a rack. A move from one domain to another changes two things: the recognition model that names an object, and the twin schema that says what its record holds. Tracking and rendering carry across untouched, so solving the spatial problem once yields reusable machinery for many industries.

Each of the settings below is the parking loop with a different recogniser and a different record.

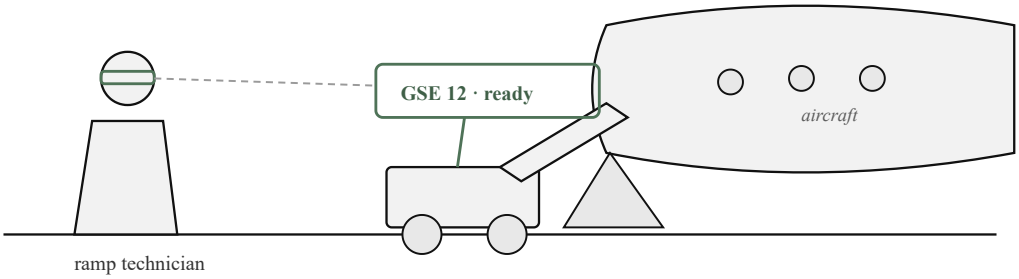


Figure 5. On an aircraft stand, a ramp technician reads the state of ground support equipment where it sits. The parking loop carries to the apron with only a change of recognised object and record.

- Airport apron work, where ground crew see aircraft, ground support equipment, and service points marked with status and turnaround timing on a busy ramp (Figure 5).
- Container yards, where an operator sees a box's manifest and dwell time held on that box among a stack of near-identical boxes (Figure 6).
- Process plants, where a technician sees a valve or pump carrying its last service date and its procedure, read at the equipment.
- Campus security, where a guard sees access points and open incidents marked along a patrol route.
- Warehouses, where a picker sees racks and totes labelled with contents and pick status among rows of identical shelving.
- Large venues, where staff see gates, sections, and safety cues marked across the site during an event.

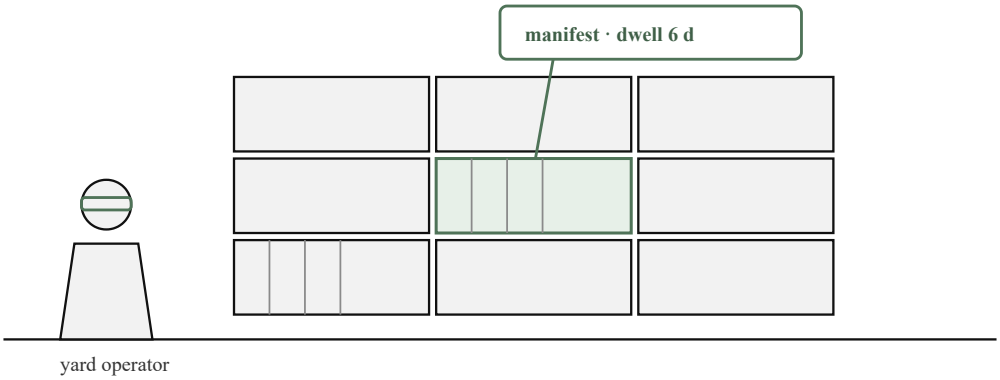


Figure 6. In a yard of near-identical boxes, one container carries its manifest and dwell time. Position singles out the right box where a printed number on a distant end-door would not.

8.1 High-density data centres

A modern data hall is a corridor of near-identical racks. Finding the unit behind an alarm means matching a label in a ticket to a spot in a long cold aisle, then confirming it by reading small identifiers on the hardware, often under time pressure while a service is degraded. The task rewards spatial marking directly. A technician walks the aisle wearing the glasses; the rack behind the alarm carries a mark in place, and the failed unit inside it carries a second. The technician reads the unit’s identity, its role, and its procedure drawn on the hardware, and logs the repair at the rack. Across a facility with thousands of identical units, binding the record to a physical position is the whole value, because a mark drawn on the unit cannot land on the wrong one.

This case sits where the capability is strongest and where satellite positioning gives nothing. A data hall is indoors, without a satellite fix, and full of repeating structure. It needs a vision-based lock on light hardware a technician can wear through a shift, with content that holds while they move down the aisle. That behaviour is what the current build shows (Figure 7).

> **WHY A REGISTERED MARK BEATS A FLOATING PANEL IN A HALL OF RACKS**
>
> A heads-up panel that floats with the gaze can show the ticket text. It cannot say which of forty identical racks the ticket points to. Only content locked to a physical position can point at the rack. In a corridor of look-alikes, a floating panel reads a label while a registered mark finds the asset.

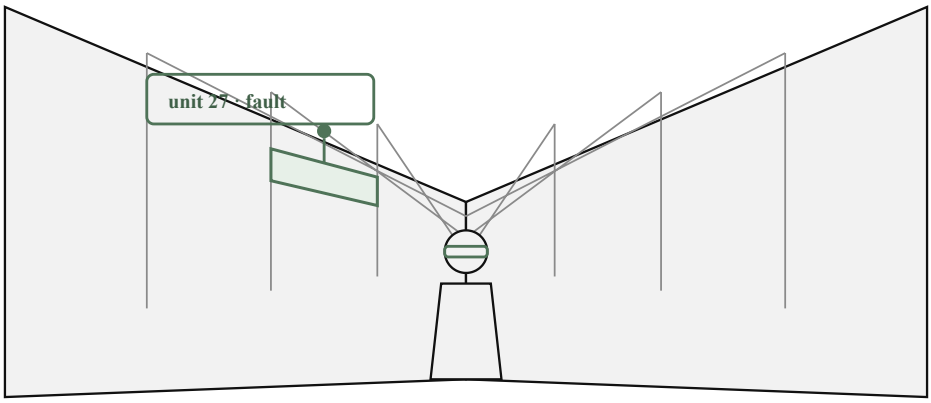


Figure 7. A technician walks a cold aisle where every rack looks alike. The unit behind an alarm is marked in place, and the search for it collapses to a glance.

9. Operational and Economic Significance

The gain from registration in operations is concrete and can be reasoned about without hype. It concerns the time crews lose to identification, the errors that identification produces, and the attention that hands-and-eyes-on-screen work takes from the task.

Identification dominates the cost at dense sites. When thousands of assets look alike, a crew spends a large share of each job matching an identifier in a work order to a physical unit, and that search grows with the count of similar assets. Drawing the record on the asset collapses the search, because the mark arrives already on the right object, and a row of assets can then be judged in one look. The same mechanism removes a common source of error: a worker who reads a handset and then turns to an asset carries an identifier in memory across the turn, and memory is where a wrong match or a mistyped entry slips in.

The value grows with the site. Small deployments see modest returns. Large infrastructure with dense, repeating assets, an airport, a container terminal, a data campus, is where identification cost runs highest and where binding records to positions returns the most. Hands-free work counts for most where stopping to read a device is unsafe or slow: on a live apron, at height, or in a cold aisle during an incident. The programme’s pilots in parking, heritage, and bounded industrial inspection were chosen to measure these gains under real conditions ahead of wider rollout.

10. Discussion

Faults in an undocumented interface separate into two classes with markedly different consequences. A fault that halts the process announces itself and admits a direct diagnosis. A fault that returns a well-formed answer does not: an ignored configuration value leaves a live position channel carrying a displacement synthesised from head rotation, a wrong argument

list returns unit-length orientations that change every frame, and an incorrect frame convention produces a scene that tracks the head convincingly. All three satisfy the checks an implementation applies by default, that a value is present, finite, and varying. The design consequence is to instrument for discrimination in place of liveness, and to build the tests that separate a correct value from a plausible one.

The frame convention shows the same principle inside the acceptance procedure. Approaching an anchored object and withdrawing from it cannot separate the convention initially adopted from the correct one, since the two agree under yaw and under forward pitch. Head roll and motion performed while facing away from the anchor distinguish them, so asymmetric motion belongs in the procedure from the outset.

Device behaviour separates along a second line, between a stationary wearer and a walking one. At rest the tracker holds its estimate and recovers it after a disturbance; under brisk movement it drifts and occasionally snaps to a fresh estimate as it relocalises. That boundary belongs to the inherited estimator, and it sets the limit of the present claim: registration that holds for a wearer working at an asset, with a known weakness during rapid transit between assets.

Removing the engine was a deliberate trade. A ready camera model, a scene graph, an asset pipeline and a frame loop were given up and paid for in code, at about three hundred lines for asset loading and a comparable amount for the camera and the draw path. Against that, the application holds a quarter of the processor share of the tracking service it depends on and occupies 497 KB, and the decisive power result came from drawing less often in place of drawing faster. On hardware this constrained, removing layers returned more than any optimisation available inside an engine.

11. Future Work

We separate the directions by maturity, so a reader never mistakes an intention for a result.

Demonstrated. Engine-free registered rendering on RayNeo X3 Pro, with the six-degree-of-freedom pose recovered from a third-party application and content that holds its place while the wearer moves around it. This is the content of the present report.

Prototype. The stereo renderer and the on-device pose loop exist as a working probe application. Four steps follow immediately. Instrument the quantities the present build leaves open, with motion-to-photon latency and a battery comparison against an equivalent engine build at the head of the list. Run a controlled comparison against the vendor’s own engine application, reproducing its reference scene in our renderer and walking the same route in the same room under both, to establish whether the drift we see under motion belongs to the estimator or to the way our client samples and consumes its output. Add a fixed per-eye offset so that near-field content carries correct depth. Evaluate pose prediction and a late-stage reprojection of the drawn frame, both of which are standard in engine and runtime compositors and absent here, as a route to steadier content without a higher frame rate.

Operational target. Vehicle and asset recognition, validation against back-office records, twin writeback with audit, and the bilingual speech layer, integrated over the demonstrated tracking for the parking pilot.

Planned. A shared coordinate frame across a wearable and a mobile robot, so a location one marks is a location the other can resolve. A flag raised by a person and a waypoint planned by a robot then name the same physical point, which turns human-robot collaboration into a concrete workflow: the robot inspects what a person marked, and a person reviews what the robot found, each in place (Figure 8).

Research direction. Semi-autonomous inspection, where a robot patrols a bounded corridor, marks anomalies at fixed points, and hands a person the marked locations for judgment. As many operators and robots write to one twin from the field, the twin becomes a live picture of a site, refreshed as crews observe, where today it holds only periodic snapshots, and a large campus gains a continuous operational view assembled from many small observations.

Speculative. Two extensions show the reach of the same loop, offered as reasoning and not as results. In a data hall, a technician points by gaze at a rack, and the system, holding registration in a corridor of identical units, resolves which one and draws its fault and its procedure on the hardware. On a street, a worker walking or driving marks potholes and surface defects at their true positions, and a later crew or an autonomous vehicle returns to the marked coordinates. Both need only a new recognition model and a new record schema; tracking and rendering carry over unchanged.

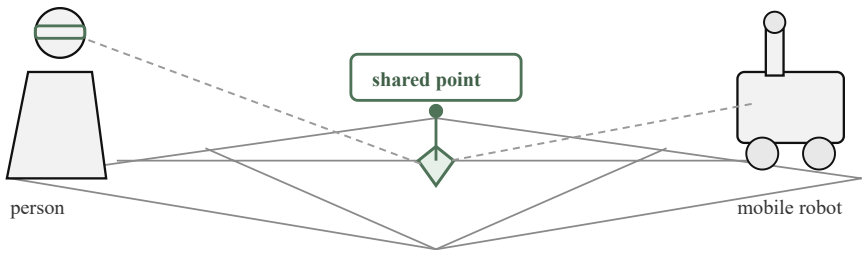


Figure 8. A person and a mobile robot resolve the same marked point from different sides. One coordinate frame lets a location set by either agent be found by the other.

12. Conclusion

Field work runs on the transfer of information between a record and the world, and registration removes that transfer by placing the record on the asset while the worker keeps both hands free. On lightweight see-through glasses the behaviour is hard to obtain, because the device

withholds head position, the panel optics defeat a first-guess calibration, the environment couples its failure modes, and the power and thermal limits are severe.

We reported a working implementation on RayNeo X3 Pro glasses. A signed third-party application recovers the head pose the interface does not publish, derives the stereo projection from the device optics, and draws registered content through a system surface with no game engine in the path. Content keeps its place in the room as the wearer circles it, and the presentation path is lean enough to fit the device and keep delay low.

The capability carries across domains without change. From parking enforcement to airport aprons to the cold aisle of a data hall, the loop is the same, and only the recognition model and the record change. Solving the one hard common problem once is why a narrow result deserves a careful report. Every field ambition in the programme reuses it.

This report describes a demonstrated engineering result and its operational context. Capabilities are labelled throughout as demonstrated, prototype, operational target, planned, research direction, or speculative. Internal methods are held back by intent.

References

1. A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, “MonoSLAM: Real-Time Single Camera SLAM,” *IEEE Trans. Pattern Anal. Mach. Intell.*, 2007. https://doi.org/10.1109/TPAMI.2007.1049
2. G. Klein and D. Murray, “Parallel Tracking and Mapping for Small AR Workspaces,” *ISMAR*, 2007. https://doi.org/10.1109/ISMAR.2007.4538852
3. R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Trans. Robotics*, 2015. https://arxiv.org/abs/1502.00956
4. C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, and J. D. Tardos, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Trans. Robotics*, 2021. https://arxiv.org/abs/2007.11898
5. A. I. Mourikis and S. I. Roumeliotis, “A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation,” *ICRA*, 2007. https://doi.org/10.1109/ROBOT.2007.364024
6. T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Trans. Robotics*, 2018. https://arxiv.org/abs/1708.03852
7. C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,” *IEEE Trans. Robotics*, 2017. https://arxiv.org/abs/1512.02363
8. J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Trans. Pattern Anal. Mach. Intell.*, 2018. https://arxiv.org/abs/1607.02565
9. Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Trans. Pattern Anal. Mach. Intell.*, 2000. https://doi.org/10.1109/34.888718

10. J. Kannala and S. S. Brandt, “A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses,” *IEEE Trans. Pattern Anal. Mach. Intell.*, 2006. https://doi.org/10.1109/TPAMI.2006.153

11. M. Tuceryan, Y. Genc, and N. Navab, “Single-Point Active Alignment Method (SPAAM) for Optical See-Through HMD Calibration,” *Presence*, 2002. https://doi.org/10.1162/105474602317473213

12. J. Grubert, Y. Itoh, K. Moser, and J. E. Swan II, “A Survey of Calibration Methods for Optical See-Through Head-Mounted Displays,” *IEEE Trans. Vis. Comput. Graph.*, 2018. https://arxiv.org/abs/1709.04299

13. R. T. Azuma, “A Survey of Augmented Reality,” *Presence*, 1997. https://doi.org/10.1162/pres.1997.6.4.355

14. E. Kruijff, J. E. Swan II, and S. Feiner, “Perceptual Issues in Augmented Reality Revisited,” *ISMAR*, 2010. https://doi.org/10.1109/ISMAR.2010.5643530

15. P. Lincoln et al., “From Motion to Photons in 80 Microseconds: Towards Minimal Latency for Virtual and Augmented Reality,” *IEEE Trans. Vis. Comput. Graph.*, 2016. https://doi.org/10.1109/TVCG.2016.2518038

16. The Khronos Group, “OpenXR Specification,” 2024. https://registry.khronos.org/OpenXR/

17. The Khronos Group, “OpenGL ES 3.2 Specification,” 2019. https://registry.khronos.org/OpenGL/specs/es/3.2/es_spec_3.2.pdf

18. Google, “Android Graphics Architecture: SurfaceFlinger, BufferQueue, and Choreographer,” Android Open Source Project documentation. https://source.android.com/docs/core/graphics

19. Google, “Android NDK Developer Guide,” 2024. https://developer.android.com/ndk/guides

20. RayNeo, “RayNeo XR OpenXR ARDK and Developer Documentation,” RayNeo Inc., developer documentation.

21. M. Grieves and J. Vickers, “Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems,” in *Transdisciplinary Perspectives on Complex Systems*, Springer, 2017. https://doi.org/10.1007/978-3-319-38756-7_4

22. F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital Twin in Industry: State-of-the-Art,” *IEEE Trans. Ind. Informatics*, 2019. https://doi.org/10.1109/TII.2018.2873186

23. A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision (CLIP),” *ICML*, 2021. https://arxiv.org/abs/2103.00020

24. A. Kirillov et al., “Segment Anything,” *ICCV*, 2023. https://arxiv.org/abs/2304.02643

25. A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision (Whisper),” 2022. https://arxiv.org/abs/2212.04356

26. Google, “ARCore Geospatial API,” developer documentation. https://developers.google.com/ar/develop/geospatial

27. Khronos Group, “glTF 2.0 Specification,” 2021. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html